Lab 5:

Build, Test, and Package Applications with IntelliJ IDEA and Docker

Name	Mehmoona Abid
Roll number	2023-BCS-097
Section	A
GitHub repository link	https://github.com/mehmoonaabid/DevOps

Getting Started

Create the screenshot directory in your existing DevOps repository:

cd ~/DevOps

mkdir -p Labs/Lab05/screenshots

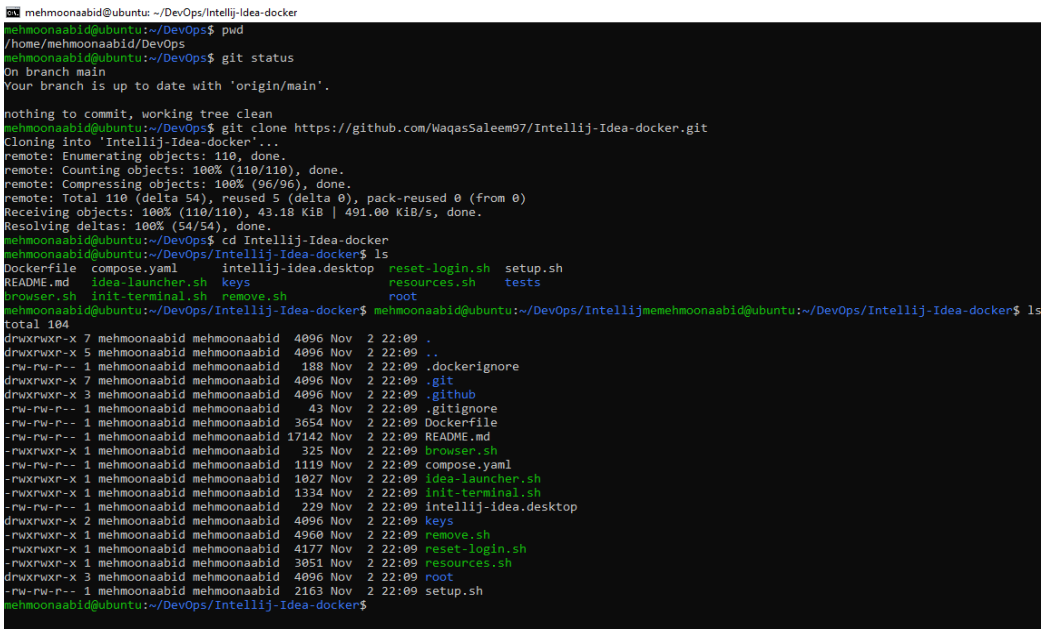
Save the images there and include them in Lab5_Solution.pdf in task order.

Task 1: Clone and Launch IntelliJ IDEA

1. In your **Ubuntu host terminal**, clone the supplied repository:
2. cd ~
3. git clone https://github.com/WaqasSaleem97/Intellij-Idea-docker.git

cd Intellij-Idea-docker

- Screenshot as intellij_repo_clone.png, showing the clone command and successful download.

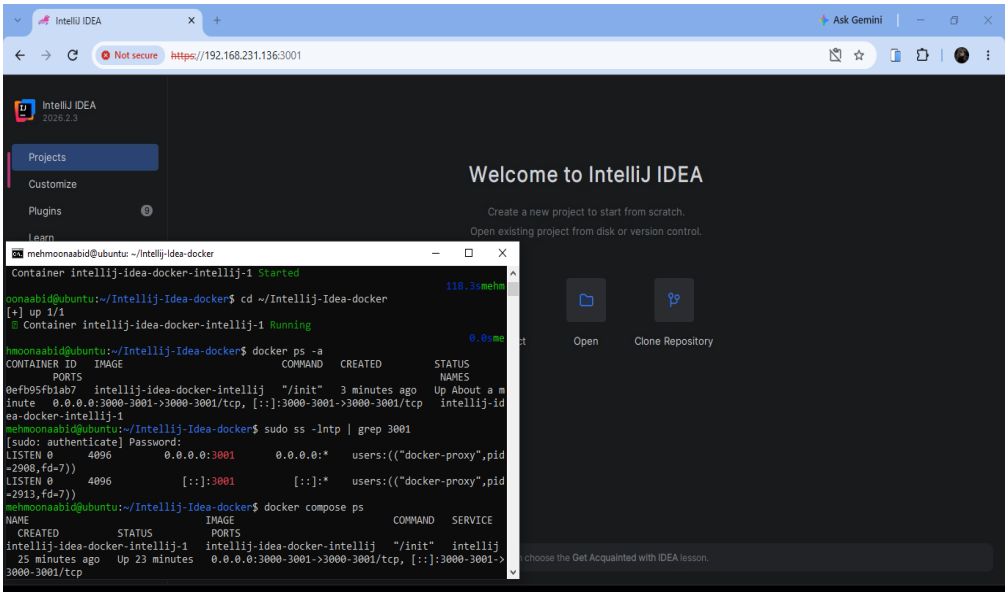


4. Build and launch the environment:
5. bash setup.sh

docker compose ps

Enter and confirm the browser password when prompted. The script builds the image and starts it with Docker Compose. The first build downloads the desktop and IDE.

- Screenshot as intellij_running.png, showing the running intellij service and desktop port 3001. Keep passwords outside the screenshot.

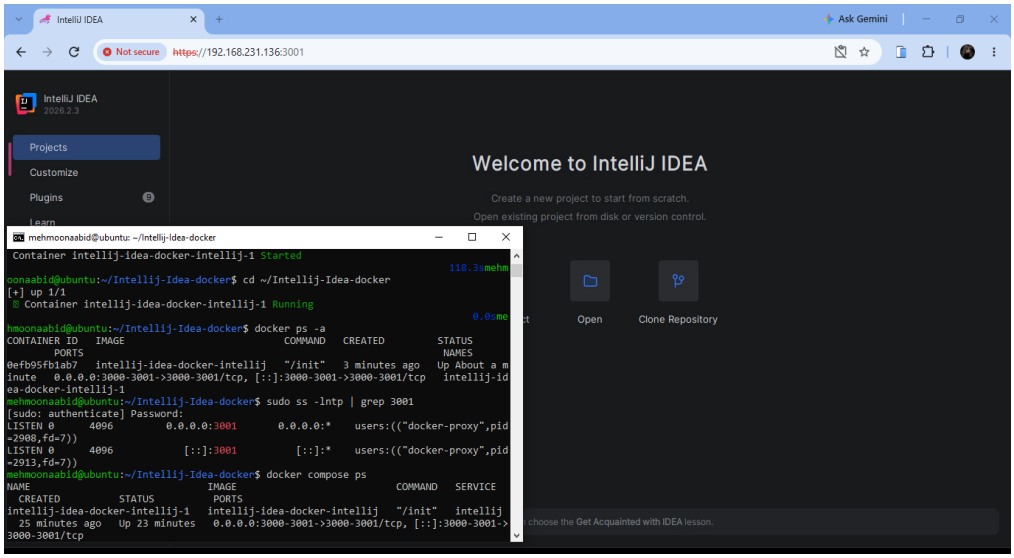


6. Open **https://localhost:3001** in your host computer's browser. Acknowledge the self-signed certificate notice for this local desktop address and sign in with browser username **admin** and the password you chose during setup.
7. Launch **IntelliJ IDEA** from the browser desktop's applications menu. If needed, open the terminal in that browser desktop and run:

idea

Complete the first-run screens and reach the IntelliJ Welcome screen.

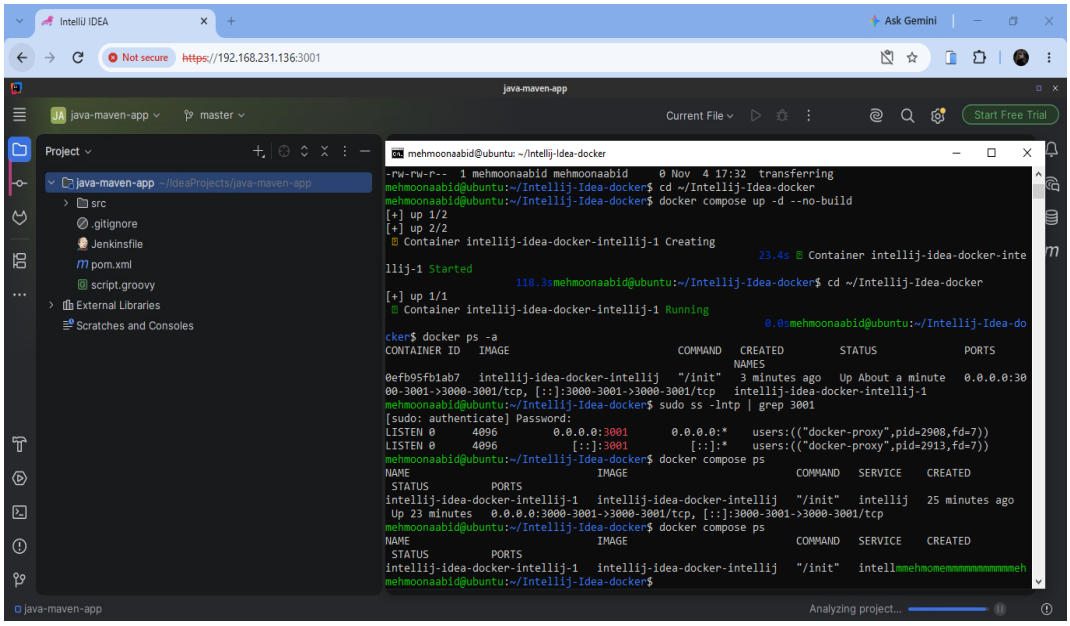
- Screenshot as intellij_desktop.png, showing IntelliJ in the browser desktop and your Ubuntu host terminal beside it.



Create all four projects under **/config/IdeaProjects**. This directory is stored in the container's persistent /config volume.

Task 2: Build and Run the Maven Project

1. On IntelliJ's Welcome screen, select **Clone Repository** or **Get from Version Control**. Enter:
    - **URL:** https://gitlab.com/nanuchi/java-maven-app.git
    - **Directory:** /config/IdeaProjects/java-maven-app
    - Screenshot as maven_project_clone.png, showing the repository URL and destination in the clone dialog.

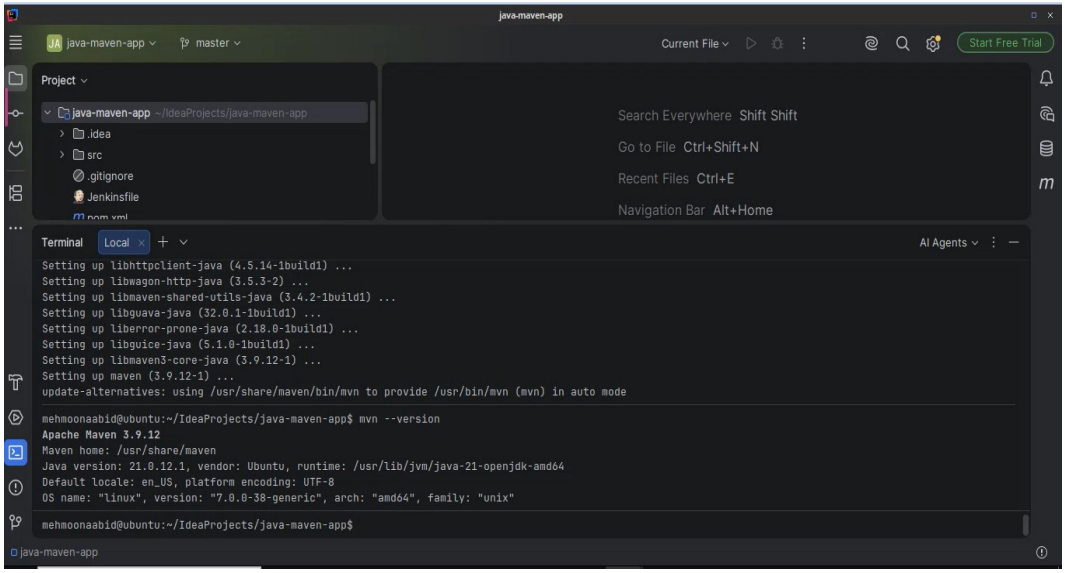


Click **Clone**, open/trust the course sample and wait for it to load. Use the Maven project configuration when asked. The project contains pom.xml.

2. Open **View → Tool Windows → Terminal** inside IntelliJ. Work from the Maven project's root directory. Run:
3. sudo apt update
4. sudo apt install -y maven

   mvn --version

   - Screenshot as maven_installed.png, showing the installed Maven version in IntelliJ's terminal.

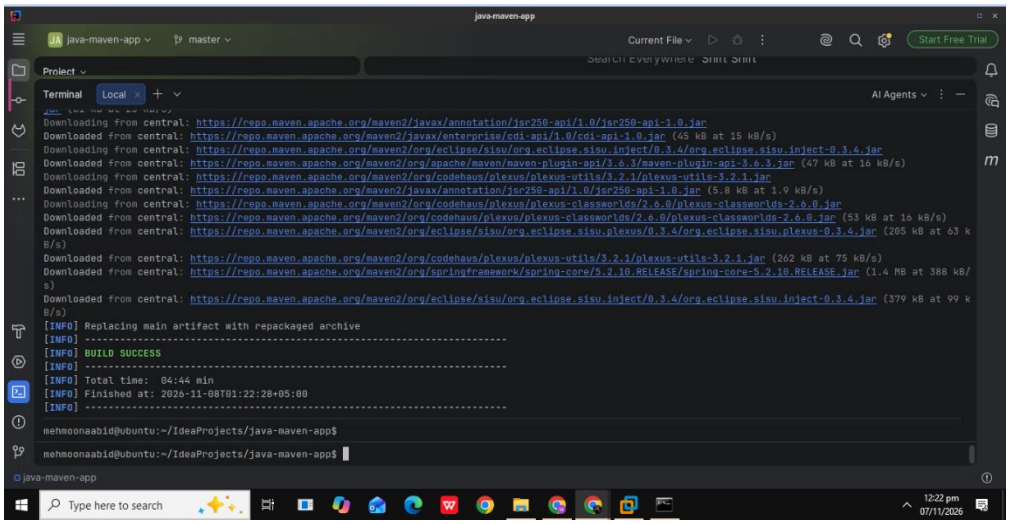


5. Package the application:

mvn package

Wait for **BUILD SUCCESS**.

- Screenshot as maven_package.png, showing the command and successful build.



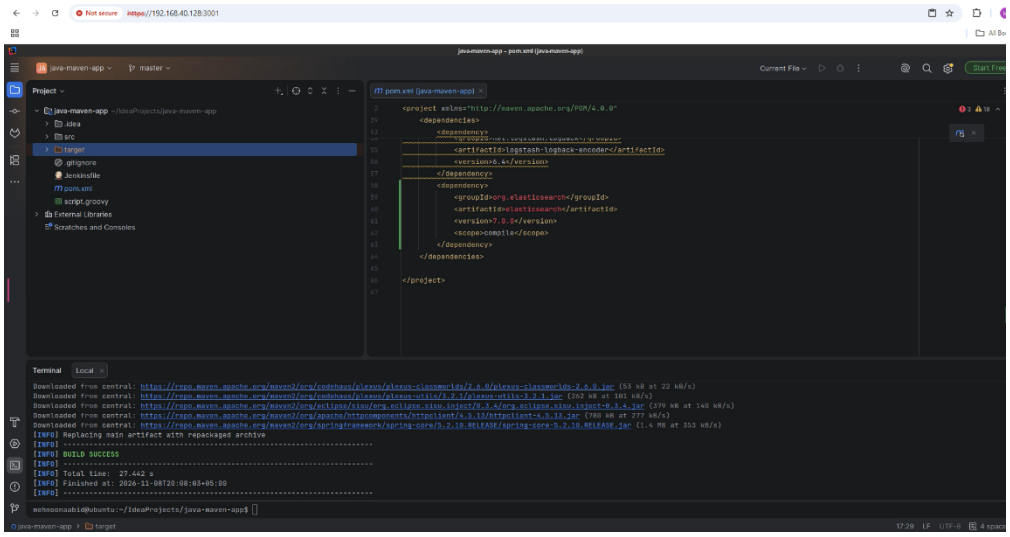
6. Open **pom.xml** in IntelliJ. Add this dependency inside the existing **<dependencies>** section, alongside the other project dependencies:

7. <dependency>
8. <groupId>org.elasticsearch</groupId>
9. <artifactId>elasticsearch</artifactId>
10. <version>7.0.0</version>
11. <scope>compile</scope>

</dependency>

Save the file. Click **Sync Maven Changes** in the editor, or open the **Maven** tool window and choose **Reload All Maven Projects**. Wait for synchronization and dependency downloads to finish. Expand **Dependencies** in the Maven tool window and locate org.elasticsearch:elasticsearch:7.0.0.

- Screenshot as maven_elasticsearch_dependency.png, showing the dependency in pom.xml and the synchronized Elasticsearch 7.0.0 entry. Keep your required terminal prompt visible.

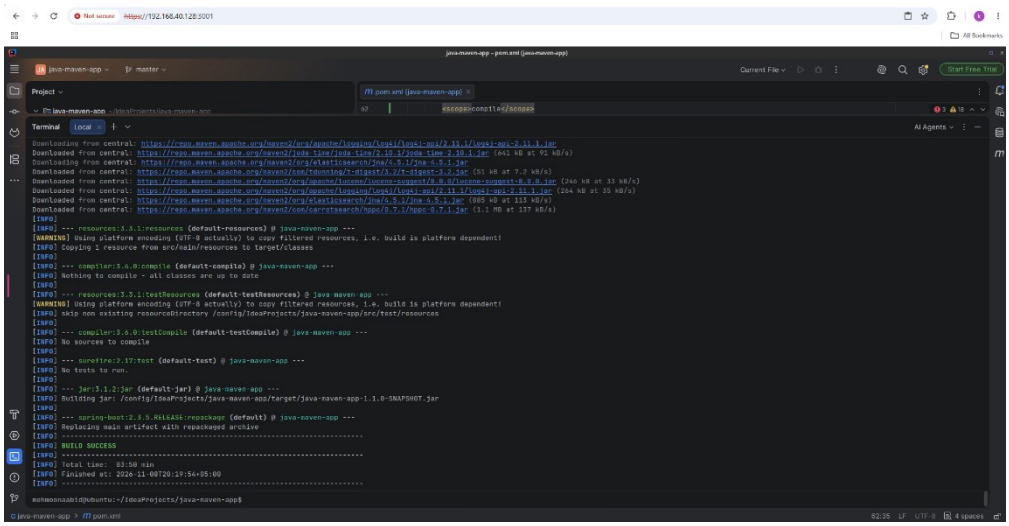


12. Package the project again after adding the dependency:

mvn package

Wait for **BUILD SUCCESS**.

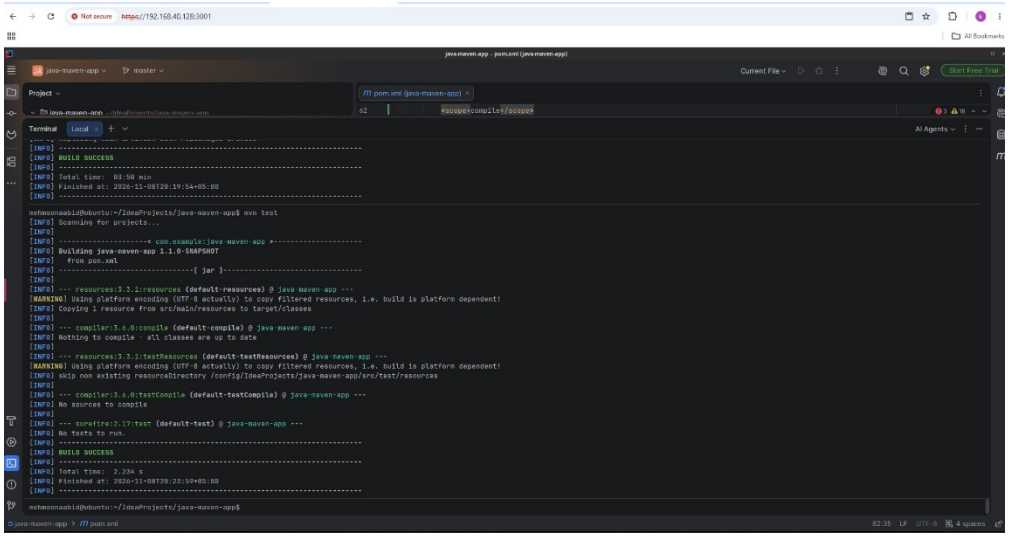
- Screenshot as maven_repackage.png, showing the command and the successful second package build.



13. Run Maven's test phase:

mvn test

- Screenshot as maven_test.png, showing the command, test output and build result.

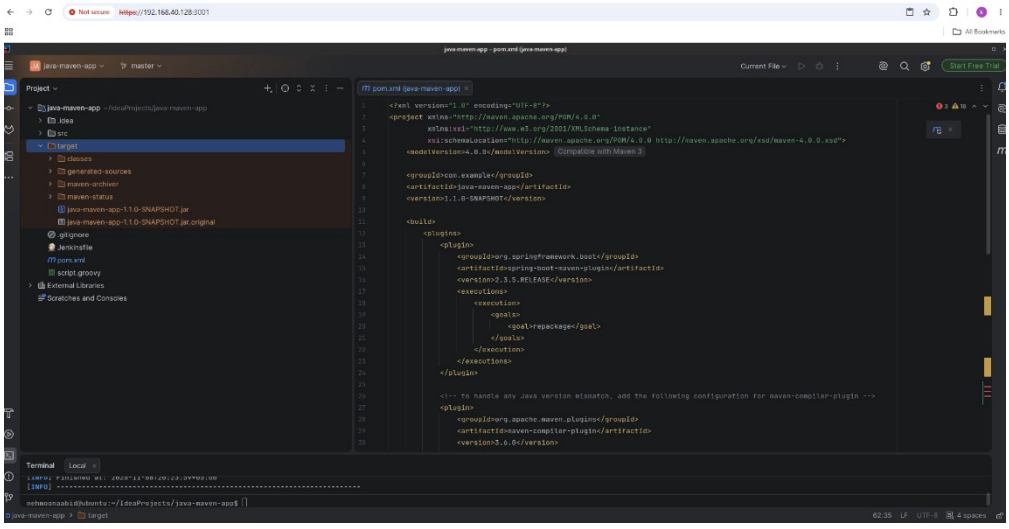


The current sample has no test source files, so Maven may report **No tests to run** with **BUILD SUCCESS**. Record that actual output; it means the test phase ran without finding tests.

14. In IntelliJ's Project pane, refresh the project if needed and expand **target**. Locate:

15. target/java-maven-app-1.1.0-SNAPSHOT.jar

    - Screenshot as maven_jar.png, showing the target directory and the exact JAR filename.

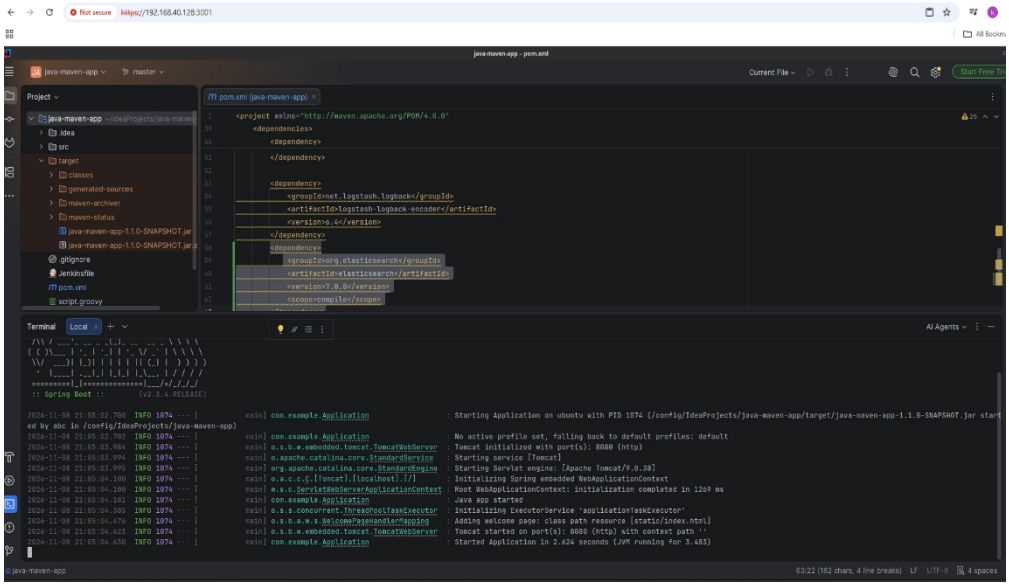


16. In the same IntelliJ terminal, run the JAR from the project root:

```
java -jar target/java-maven-app-1.1.0-SNAPSHOT.jar
```

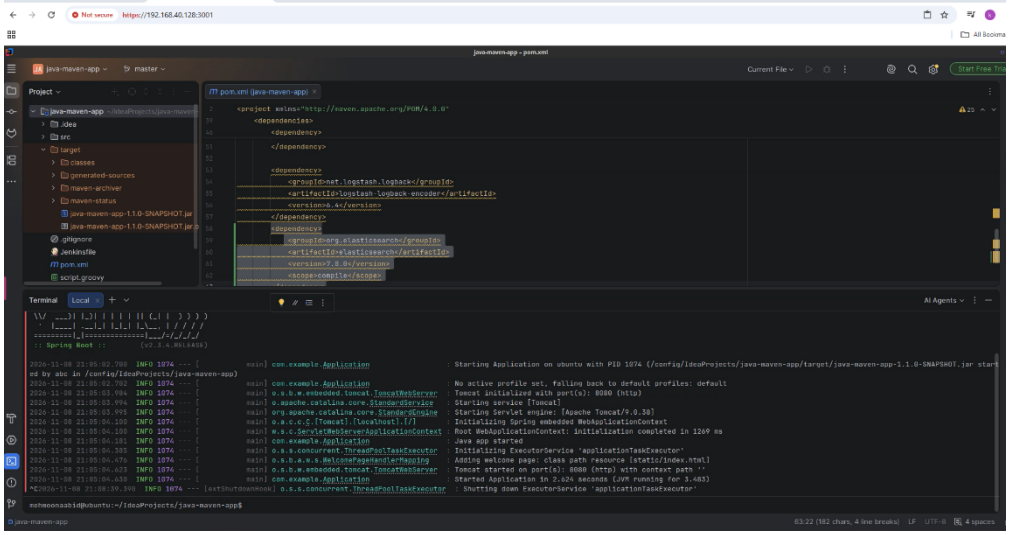
Wait for the Spring Boot/Tomcat startup output. The sample runs as a web application on port 8080.

- Screenshot as maven_running.png, showing the JAR command and successful application startup.



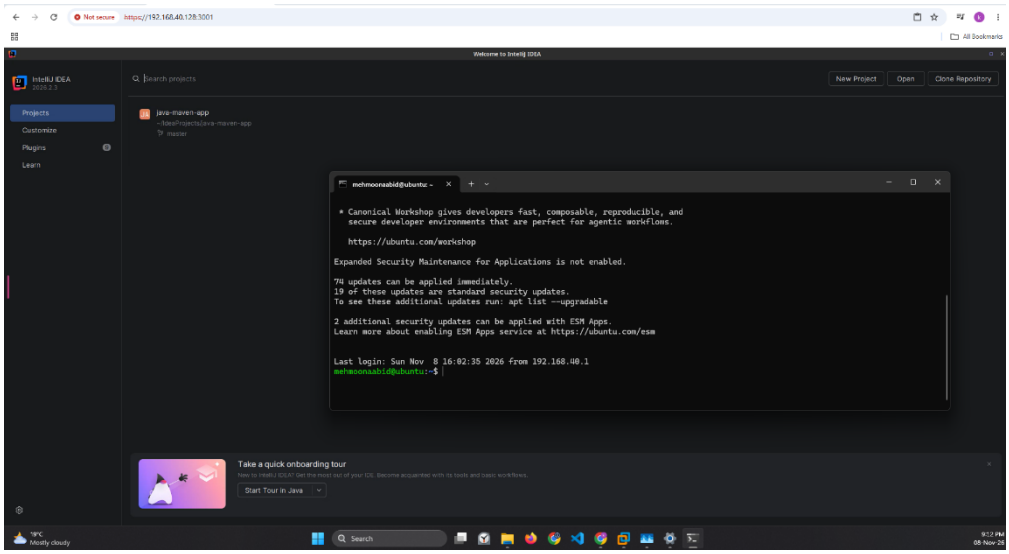
17. Press **Ctrl+C** in that terminal to stop the application. Wait for the shell prompt to return.

- Screenshot as maven_stopped.png, showing ^C/shutdown output and the returned prompt.



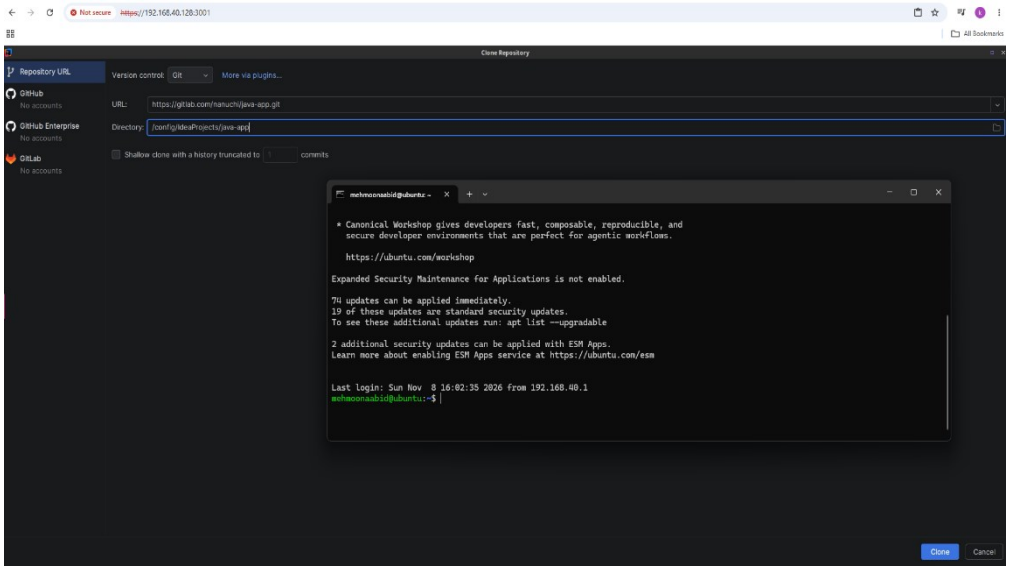
18. Select **File → Close Project** and return to IntelliJ's Welcome screen.

    - Screenshot as maven_project_closed.png. Use your Ubuntu host terminal beside the browser for identity evidence.



Task 3: Build and Run the Gradle Project

1. From IntelliJ's Welcome screen, use **Clone Repository / Get from Version Control** again:
    - **URL:** https://gitlab.com/nanuchi/java-app.git
    - **Directory:** /config/IdeaProjects/java-app
    - Screenshot as gradle_project_clone.png, showing the URL and destination.



Click **Clone**, open/trust the sample and load it as a Gradle project. It contains build.gradle, settings.gradle and gradlew.

2. Select the project's **Gradle Wrapper** when IntelliJ asks which Gradle installation to use.
3. Configure JDK 16 before running the Gradle Wrapper:
    - Select **File → Project Structure → Project**.
    - Set **Project SDK** to **JDK 16**.

- Set **Project language level** to **8 - Lambdas, type annotations, etc.**
- If JDK 16 is unavailable, open the SDK list and select **Download JDK**. Use a JDK build and architecture compatible with your computer.

The image above is an instructor reference. Capture your own project and build evidence.

Then open **File → Settings → Build, Execution, Deployment → Build Tools → Gradle** and set **Gradle JVM** to **Project SDK (JDK 16)** or the downloaded JDK 16. Select **Apply** and **OK**, then close and reopen IntelliJ's terminal.

The supplied wrapper is Gradle 7.x. It can run on Java 16, but it cannot run on Java 21. Java 21 produces class-file major version 65 and requires Gradle 8.5 or newer to run Gradle itself.

4. Open **View → Tool Windows → Terminal**. From the Gradle project root, verify the JVM used by the wrapper:

```
./gradlew --version
```

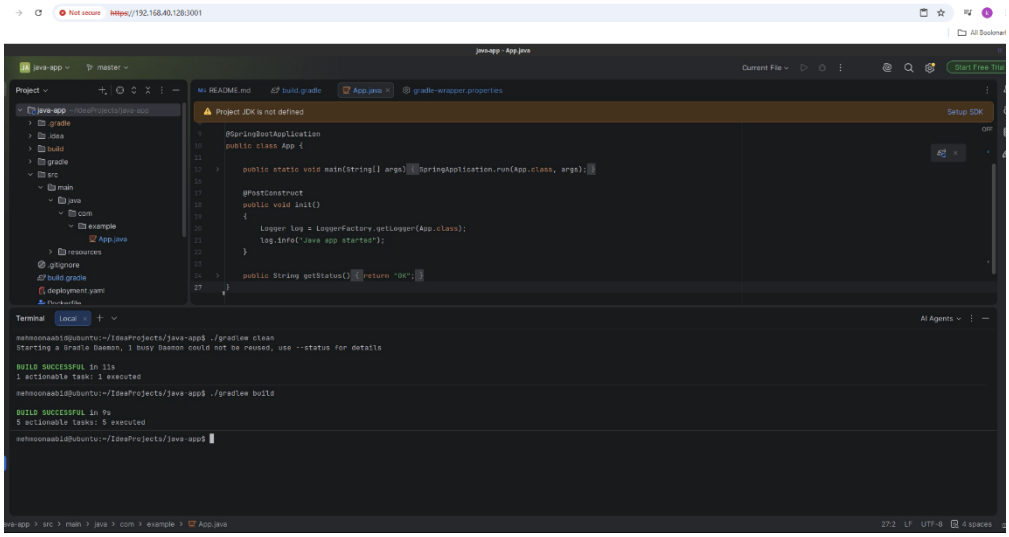
Confirm that the JVM information reports Java **16**. Do not continue while it reports Java 21.

Build the project:

```
./gradlew build
```

The wrapper downloads the project's Gradle version on its first run. Wait for **BUILD SUCCESSFUL**.

- Screenshot as gradle_build.png, showing the command and successful build.



If the build reports **Unsupported class file major version 65**, Gradle is still running on Java 21. Recheck both the Project SDK and Gradle JVM, reopen the terminal, confirm Java 16 with ./gradlew --version, and retry.

If the wrapper reports Permission denied, run chmod +x gradlew and retry ./gradlew build.

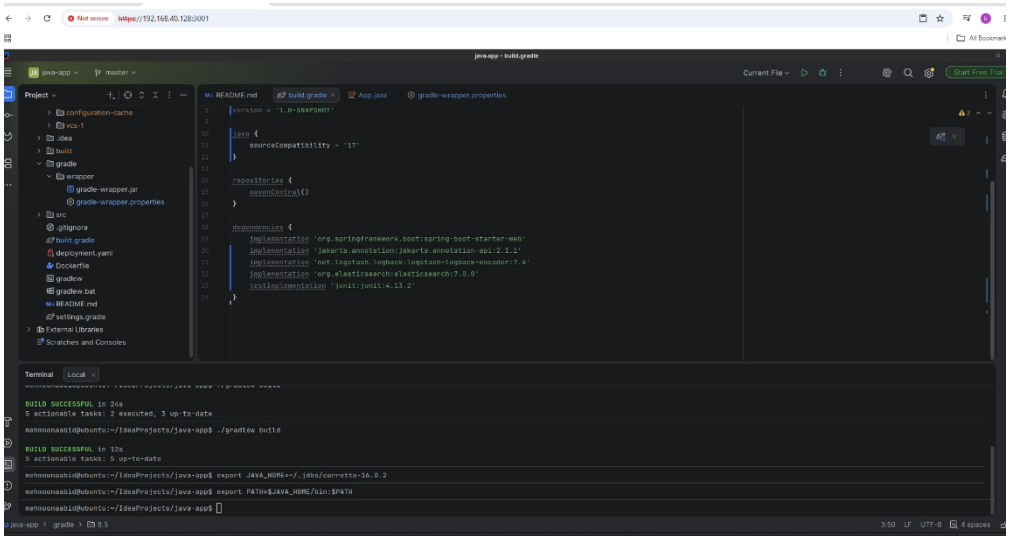
5. Open **build.gradle** and add this line inside the existing **dependencies { ... }** block:

implementation("org.elasticsearch:elasticsearch:7.0.0")

Keep the other dependencies. Save the file and click **Sync Gradle Changes**, or open the **Gradle** tool window and choose **Sync All Gradle Projects**. Wait for synchronization to finish, then locate Elasticsearch 7.0.0 under the project's **External Libraries**.

- Screenshot as gradle_elasticsearch_dependency.png, showing the dependency in build.gradle and

the synchronized Elasticsearch library. Keep your required terminal prompt visible.

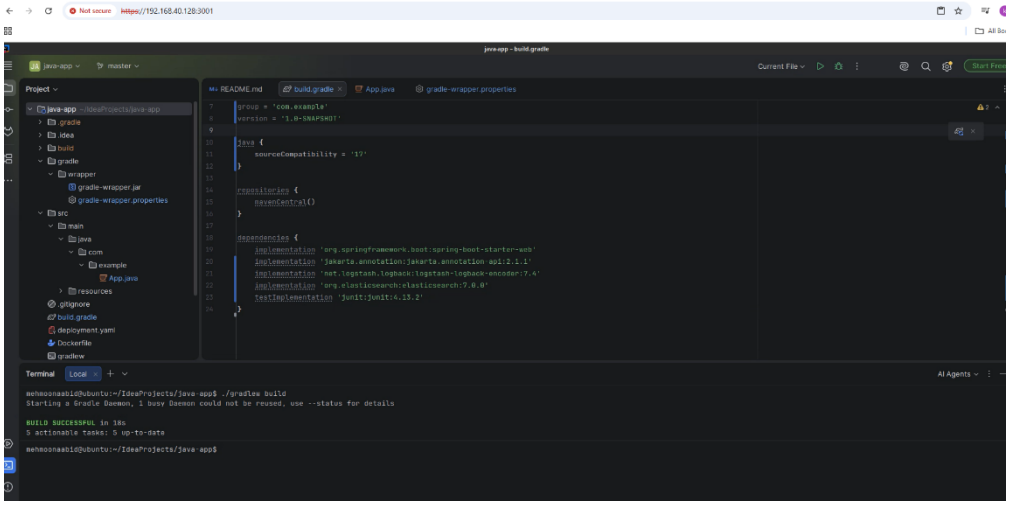


6. Build the project again after the dependency change:

./gradlew build

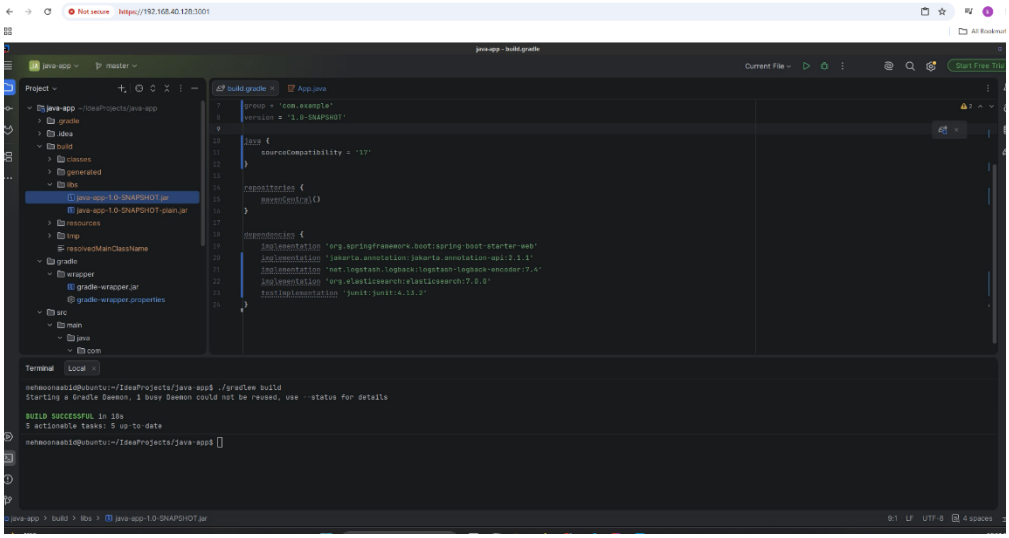
Wait for **BUILD SUCCESSFUL**.

- Screenshot as gradle_rebuild.png, showing the command and successful rebuild.



7. In IntelliJ's Project pane, expand **build → libs**. Locate:

8. build/libs/java-app-1.0-SNAPSHOT.jar
    - Screenshot as gradle_jar.png, showing the output directory and exact JAR filename.

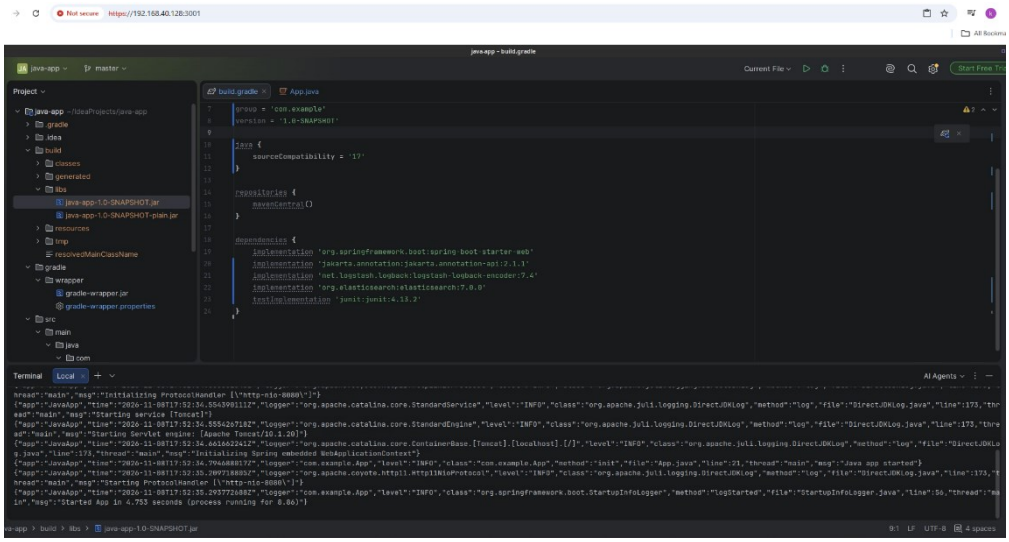


9. Run the generated JAR from the project root:

java -jar build/libs/java-app-1.0-SNAPSHOT.jar

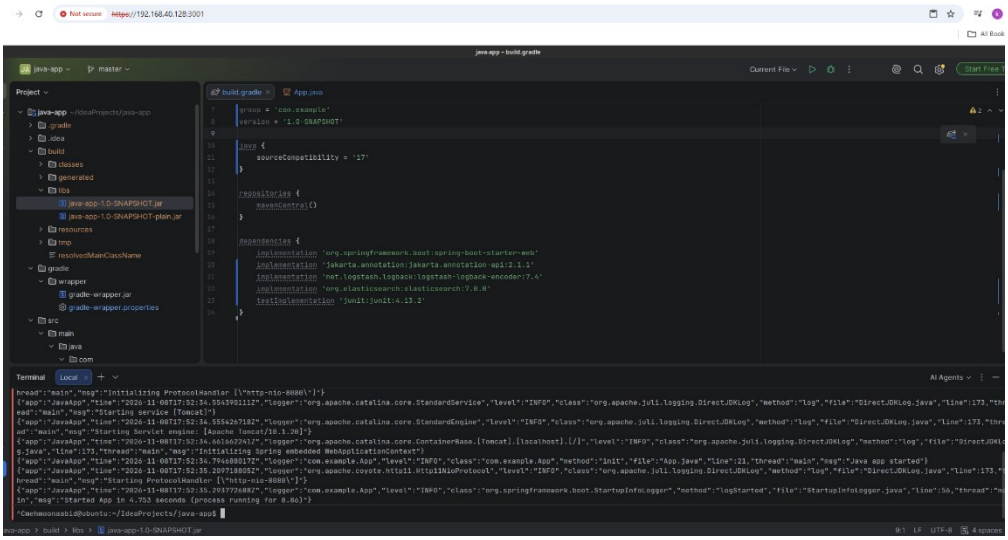
Wait for successful application startup.

    - Screenshot as gradle_running.png, showing the command and startup output.

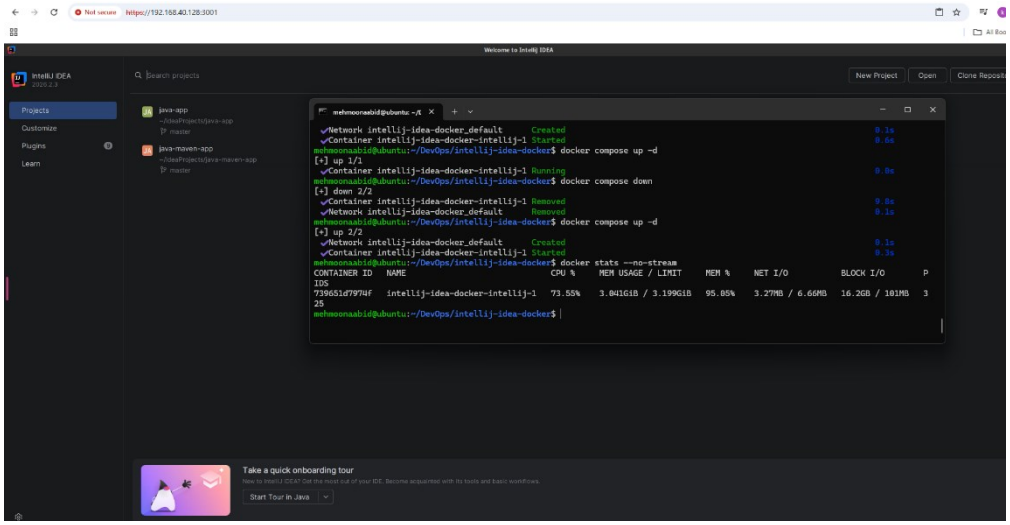


10. Press **Ctrl+C** and wait for the prompt to return. This also releases port 8080 before another application uses it.

- Screenshot as gradle_stopped.png, showing the shutdown and returned prompt.

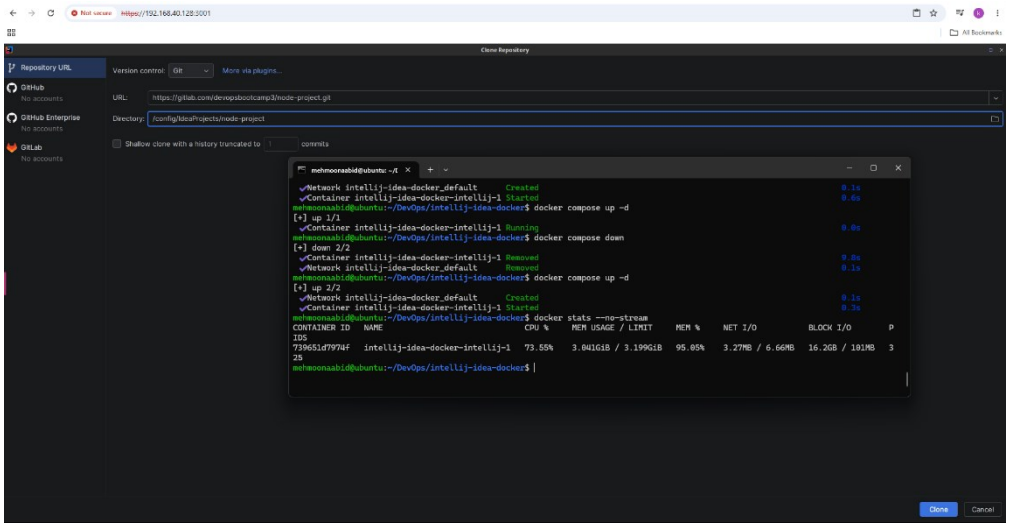


11. Select **File → Close Project**.
    - Screenshot as gradle_project_closed.png, showing the Welcome screen beside your Ubuntu host terminal.



Task 4: Install and Run the Node.js Project

1. Use IntelliJ's **Clone Repository / Get from Version Control** action:
    - **URL:** https://gitlab.com/devopsbootcamp3/node-project.git
    - **Directory:** /config/IdeaProjects/node-project
    - Screenshot as node_project_clone.png, showing the URL and destination.

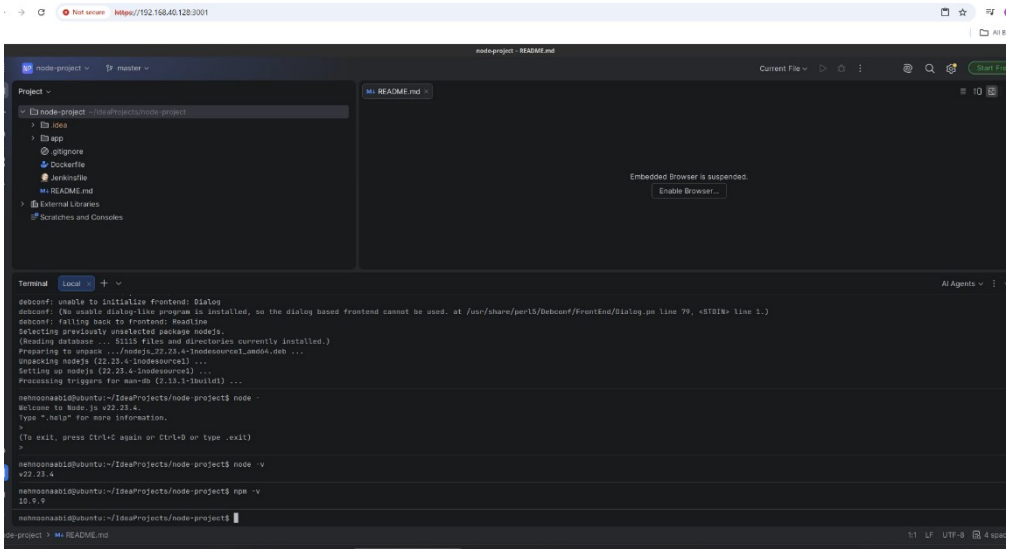


Clone and open the project. Its application files are in the app subdirectory.

2. Open the terminal **inside IntelliJ** and install Node.js together with npm:
3. sudo apt update
4. sudo apt install -y nodejs npm
5. node --version

npm --version

- Screenshot as nodejs_installed.png, showing both installed versions.

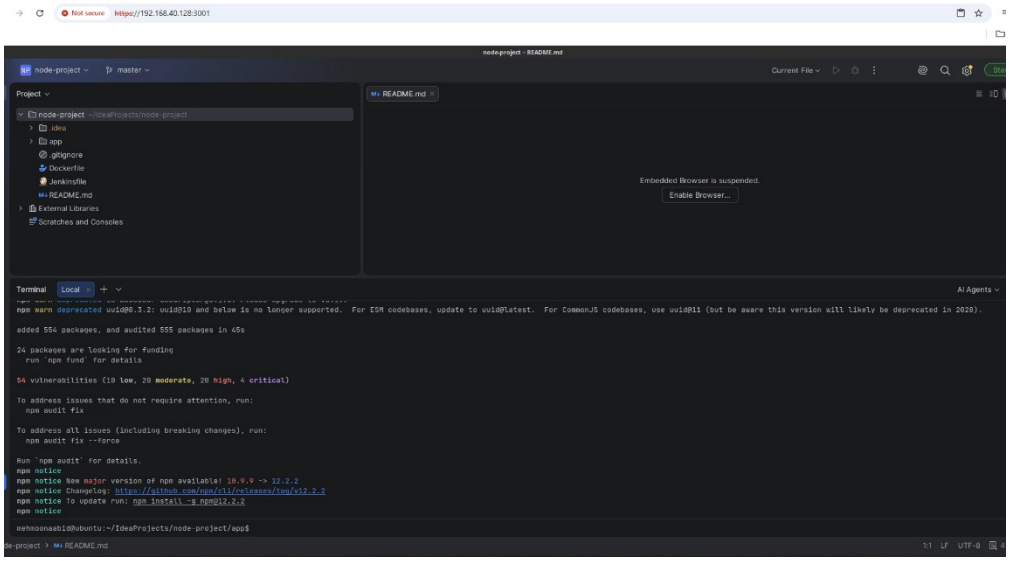


6. From the project root, enter app and install its dependencies:
7. cd app

npm install

The app/package.json file defines the dependencies and the start script.

- Screenshot as npm_install.png, showing the command and completed installation.



8. Start the application:

npm start

The supplied server.js initially listens on port 3000. Webtop also uses that port inside this container, so the initial start may fail with **EADDRINUSE**.

9. If port 3000 fails, press **Ctrl+C** if the process is still active. Open **app/server.js** in IntelliJ and change the listener and its message to:

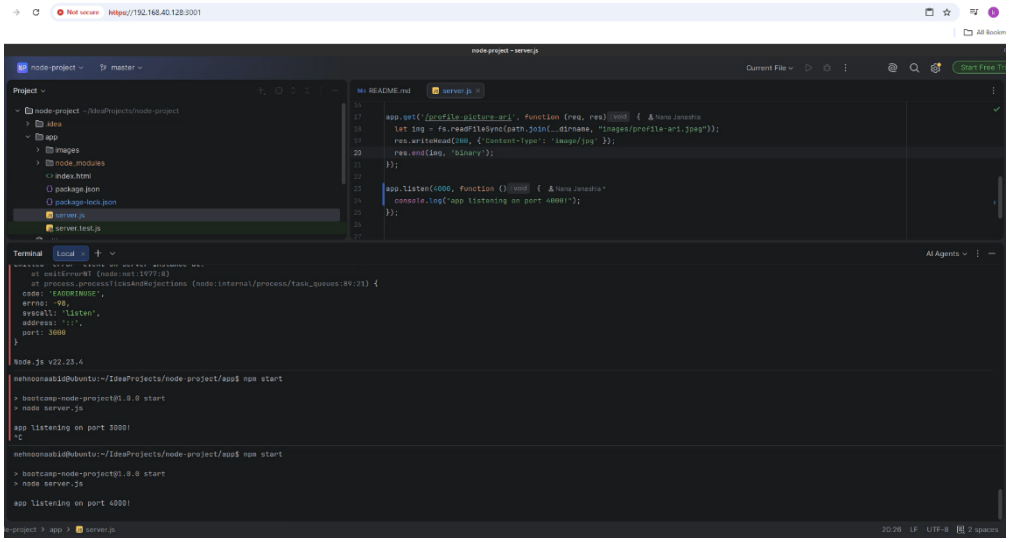
10. app.listen(4000, function () {

11. console.log("app listening on port 4000!");

});

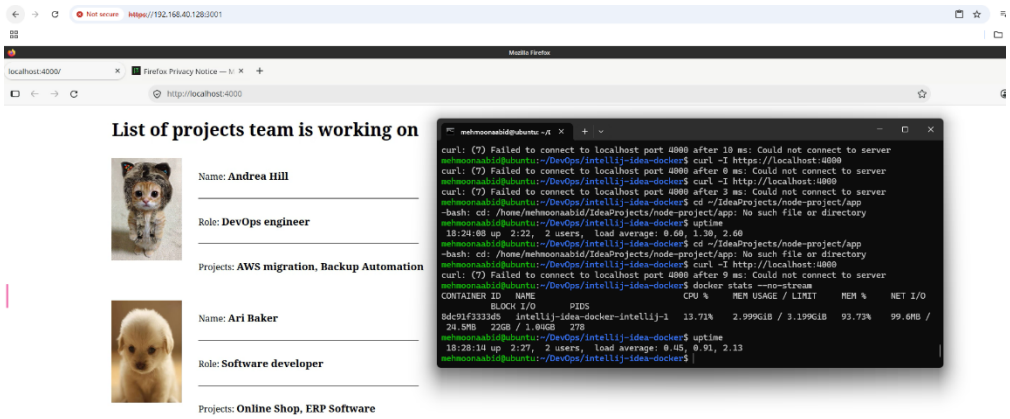
Save the file and run npm start again from the app directory. Use port **4000** for the final browser check inside the container desktop. Keep Webtop running while changing the application port.

- Screenshot as node_running.png, showing npm start and app listening on port 4000!.



12. Keep the application running. Open a browser **inside the container's desktop** from its applications menu and visit http://localhost:4000. Verify that the Node.js project page and its profile images are visible.

    - Screenshot as node_browser.png, showing the page, its URL and your IntelliJ terminal in a split-screen view inside the container desktop.



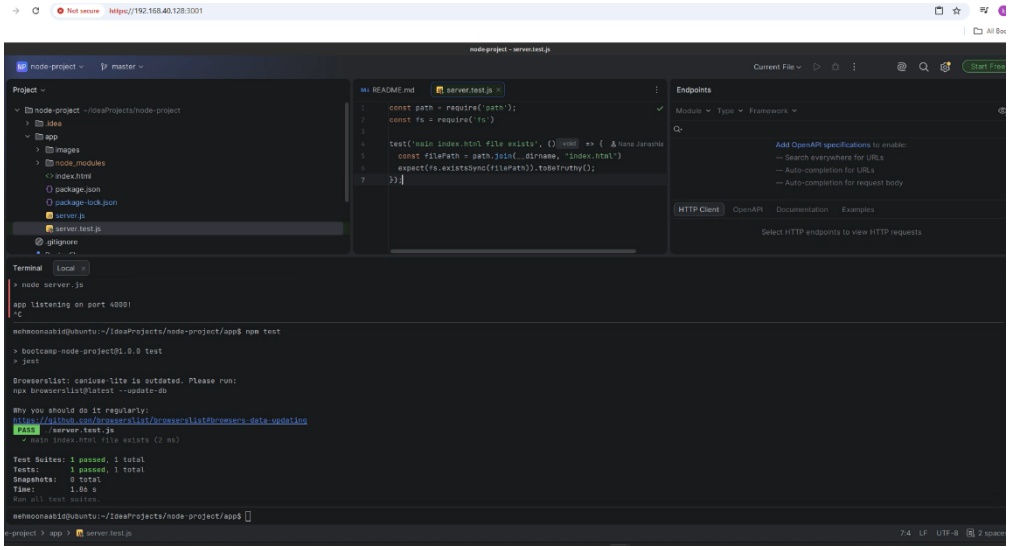
13. After saving the evidence, stop the Node.js application with **Ctrl+C** in IntelliJ's terminal.

14. Stay in the **app** directory and run the application's tests:

```
npm test
```

The project's test script runs Jest. The supplied test checks that index.html exists. Wait for the test result.

- Screenshot as npm_test.png, showing the command and passing test summary.

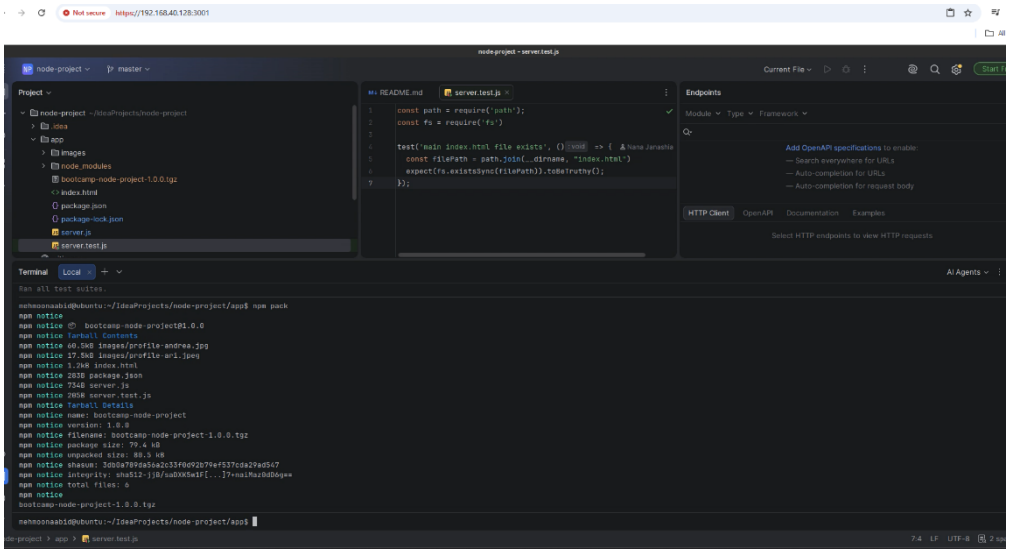


15. Create the application's package artifact:

```
npm pack
```

npm uses the name and version in package.json to create **bootcamp-node-project-1.0.0.tgz** in the app directory. This tarball packages the application's files for distribution.

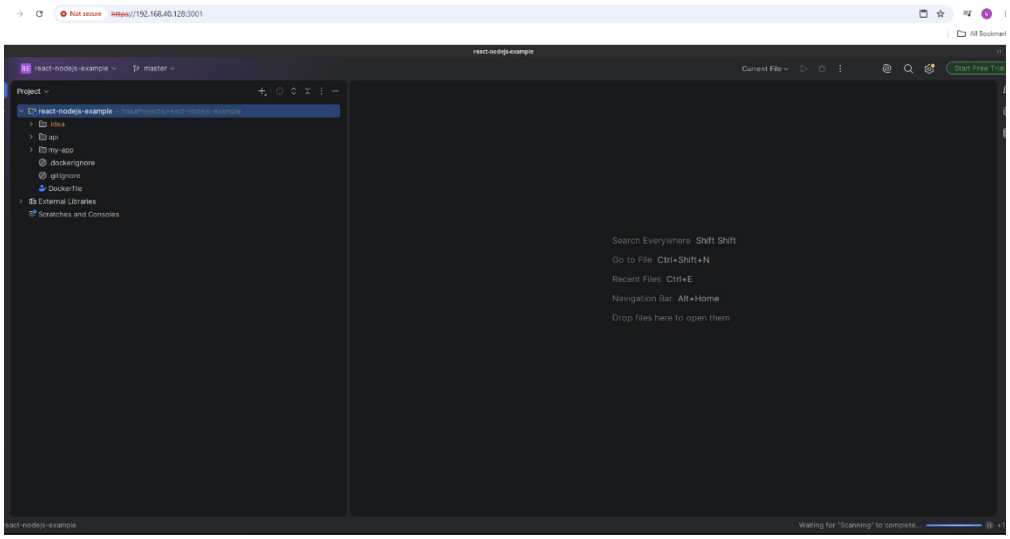
- Screenshot as npm_pack.png, showing the command and generated .tgz filename.



16. Select **File → Close Project** to return to the Welcome screen. Keep this clone for Task 6.

Task 5: Build the React and Node.js API Artifact

1. From IntelliJ's Welcome screen, use **Clone Repository / Get from Version Control**:
    - **URL:** https://github.com/bbachi/react-nodejs-example.git
    - **Directory:** /config/IdeaProjects/react-nodejs-example
    - Screenshot as react_node_project_clone.png, showing the repository URL and destination.



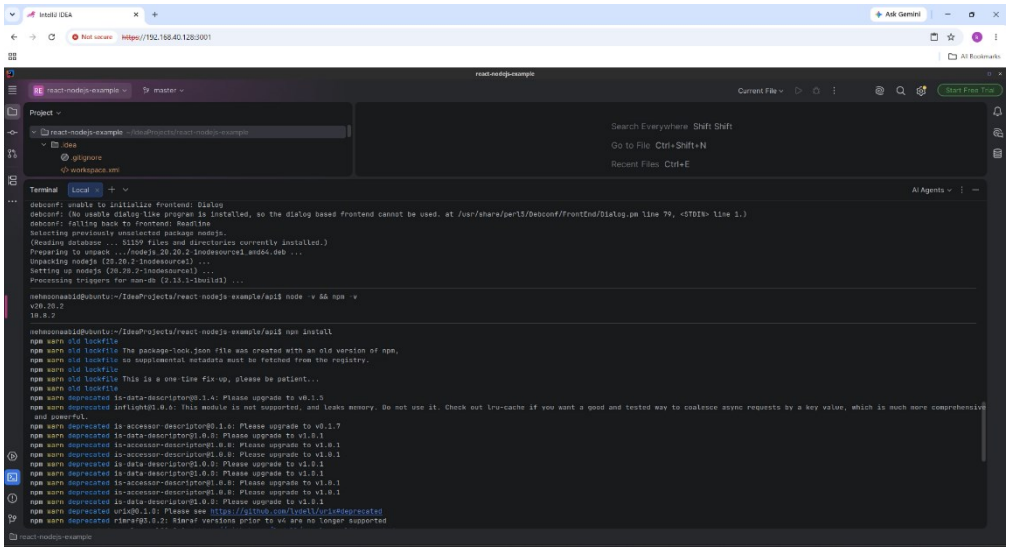
Clone and open the project. This task builds the Node.js API in its api directory.

2. Open the terminal **inside IntelliJ**. From the project root, run:
3. cd api

npm install

After installation, refresh the Project pane if needed and expand **api**. Locate **node_modules**, which contains the installed dependencies.

- Screenshot as react_api_dependencies.png, showing the completed npm install and the api/node_modules directory.



4. In the same api directory, build the API artifact:

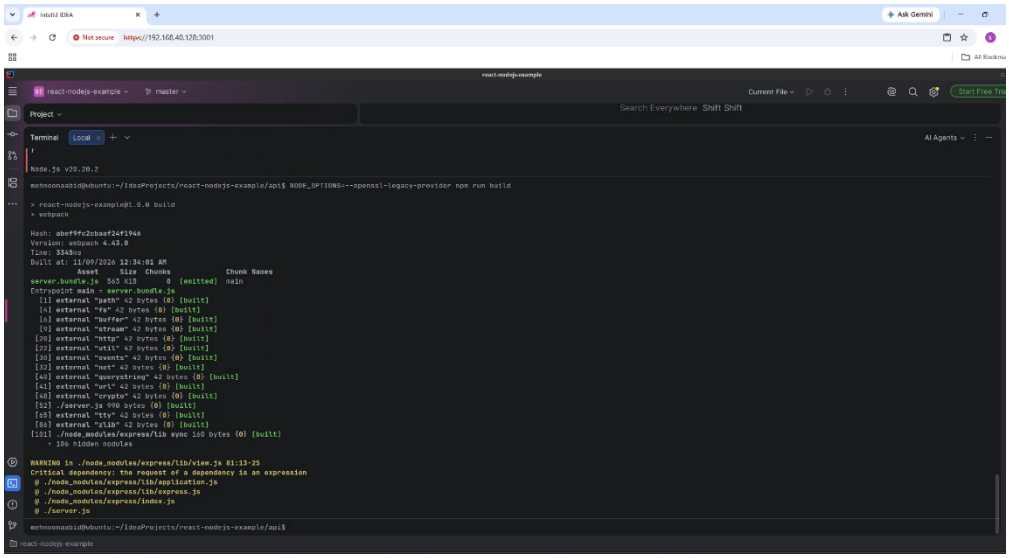
npm run build

The build script runs webpack. Its configuration writes the bundled application to **api/server.bundle.js**, replacing the existing bundle if one is already present.

If this older webpack sample reports **ERR_OSSL_EVP_UNSUPPORTED** on your Node.js version, retry this build with the compatibility option for that command:

NODE_OPTIONS=--openssl-legacy-provider npm run build

- Screenshot as react_api_build.png, showing the successful build command/output and server.bundle.js in the api directory.

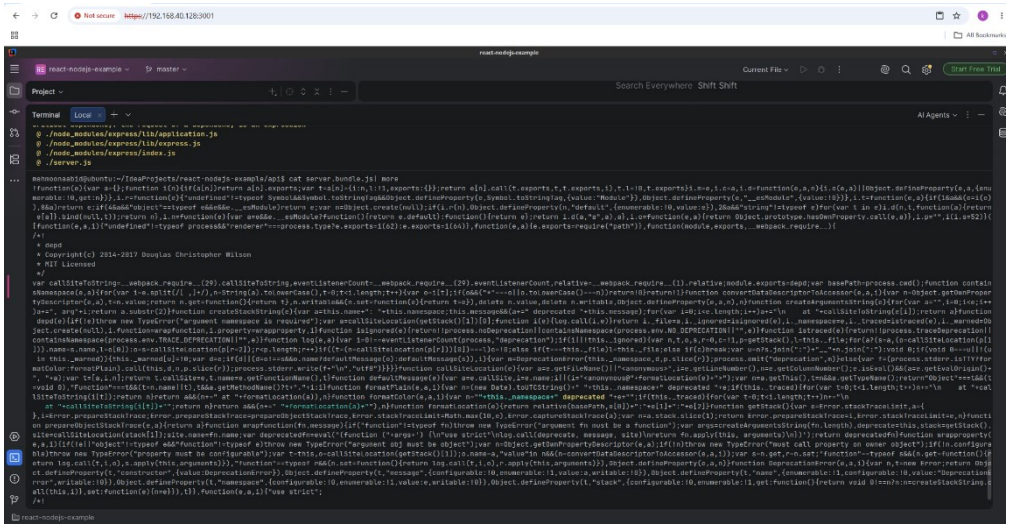


5. View the generated bundle:

cat server.bundle.js | more

Press **Space** to move through its contents.

- Screenshot as react_api_bundle.png, showing the viewing command and bundled JavaScript. Use a split terminal view to keep your required identity visible while the pager is open



6. Press **q** to quit the viewer and return to the terminal.
   Select **File → Close Project**.

Task 6: Build a Docker Image for the Node.js Project

1. In IntelliJ, choose **Open** from the Welcome screen, or **File → Open**, and reopen the existing **/config/IdeaProjects/node-project** clone from Task 4:
    - **Repository:** https://gitlab.com/devopsbootcamp3/node-project.git

Open IntelliJ's terminal and move to the **repository root**:

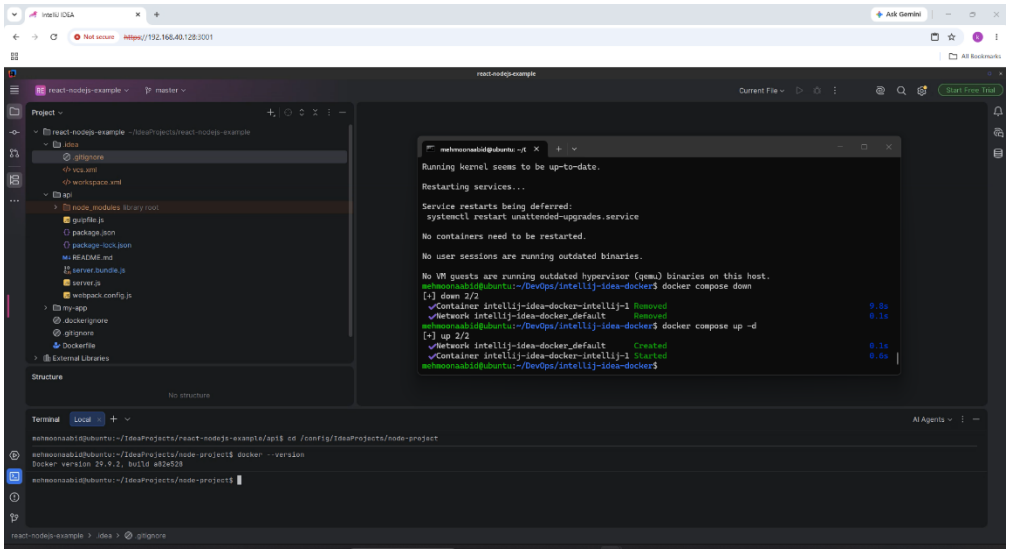
cd /config/IdeaProjects/node-project

The Dockerfile is in this directory, alongside the app folder. Run all commands in this task from this root directory.

2. Check that the Docker command is installed:

docker --version

    - Screenshot as docker_installed.png, showing the command and installed version.

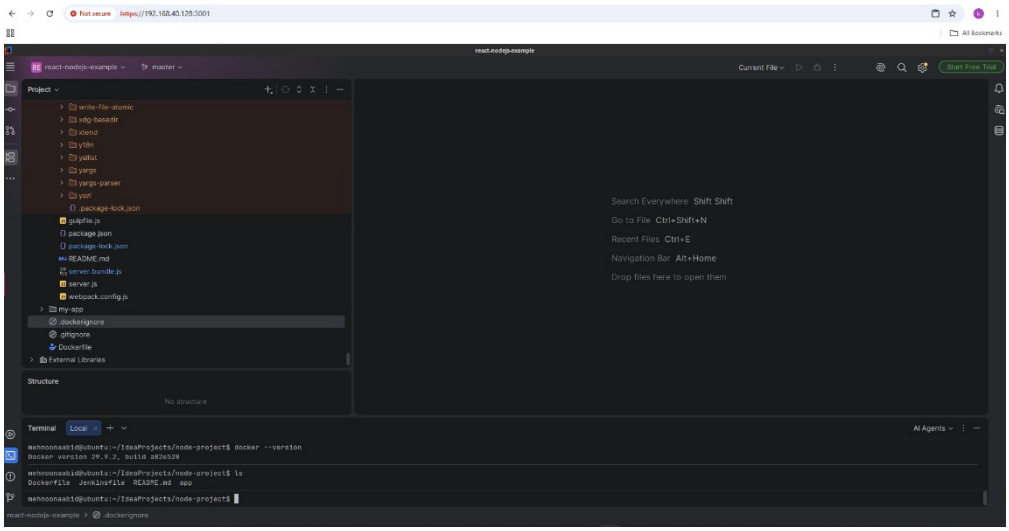


The current IntelliJ setup repository shares the Ubuntu host's Docker socket with the desktop container. The following sudo docker commands use that host's Docker Engine. If you are using an older environment and see a missing /var/run/docker.sock error, apply the existing-installation update in the IntelliJ repository instructions from your Ubuntu host before continuing.

3. List the repository files and locate **Dockerfile**:

   ls

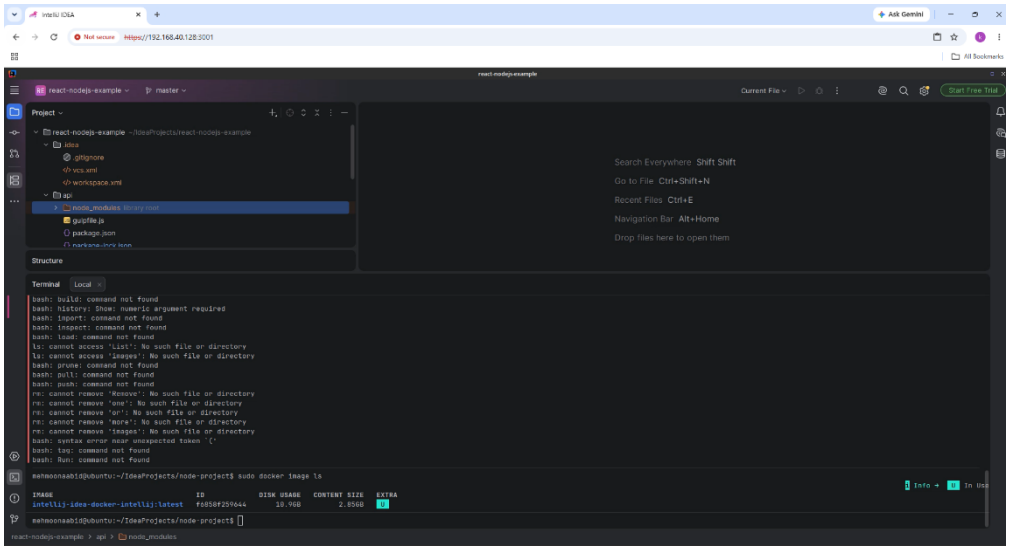
   - Screenshot as dockerfile_exists.png, showing the command and Dockerfile in the output.



4. View the existing Docker images before building:

sudo docker images

- Screenshot as docker_images_before.png, showing the command and current image list.



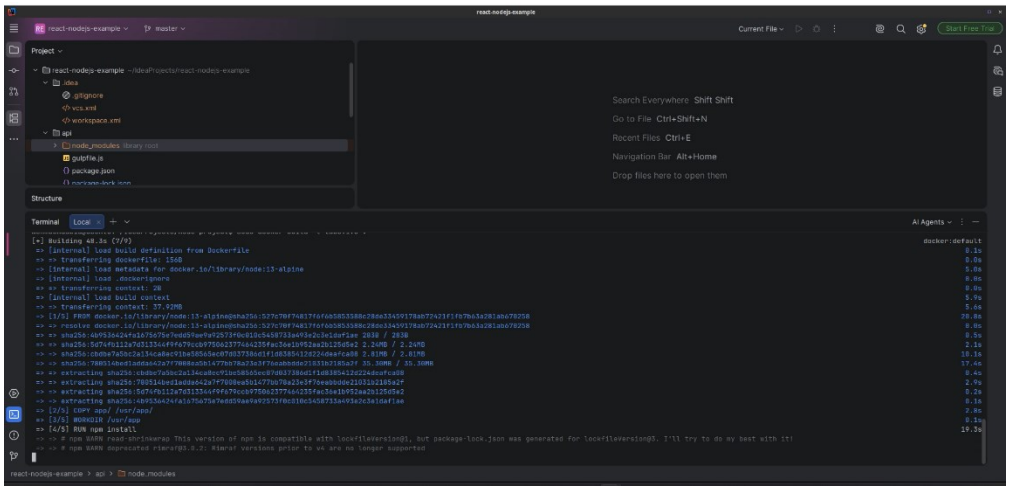
5. Build the application image:

sudo docker build -t lab5:1.0 .

The final **.** uses the current repository root as the build context. Docker reads its Dockerfile, copies the app directory, installs dependencies and runs the supplied npm

test during the image build. Wait for the build to finish successfully.

- Screenshot as docker_build.png, showing the build command and successful image build/tag output.

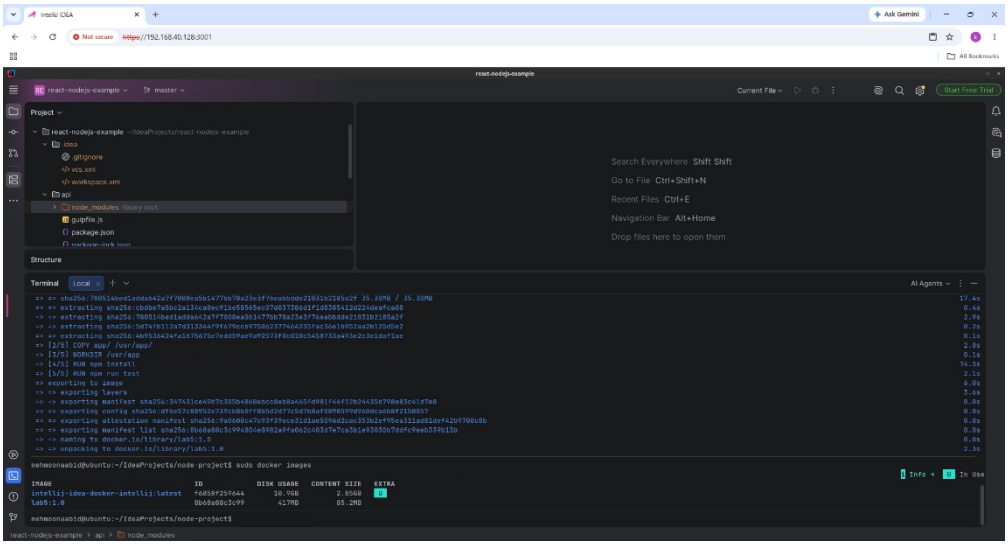


6. List the images again:

sudo docker images

Find **lab5** in the **REPOSITORY** column and **1.0** in the **TAG** column. Together they identify **lab5:1.0**.

- Screenshot as docker_images_after.png, showing the command, image repository, tag and image ID.


IntelliJ IDEA
Ask Gemini
Not secure https://192.168.40.128:3001
react-nodejs-example
react-nodejs-example
master
Current File
Start Free Trial
Project
react-nodejs-example ~/IdeaProjects/react-nodejs-example
idea
.gitignore
vcs.xml
workspace.xml
api
node_modules library root
gulpfile.js
package.json
Search Everywhere Shift Shift
Go to File Ctrl+Shift+N
Recent Files Ctrl+E
Navigation Bar Alt+Home
Drop files here to open them
Structure
Terminal
Local
AI Agents
mehmoonaabid@ubuntu:~/IdeaProjects/node-project$ sudo docker images
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
intellij-idea-docker-intellij:latest f6038f259444 10.9GB 2.85GB U
lab5:1.0 0b68a08c3c99 417MB 85.2MB
mehmoonaabid@ubuntu:~/IdeaProjects/node-project$
react-nodejs-example > api > node_modules